

COMP101

Introduction to Programming

Tutorial-03 Task Solution

Problem Specification

Rough Statement

- The initial request for a programmable solution to a problem is often presented as a 'rough statement'
- This cannot form the basis for a programmed solution
 - Gives a general idea of the problem domain
 - Usually contains omissions, ambiguities
- A '**problem specification**' refines the rough statement
 - deal with omissions, inconsistencies, ambiguities and uncertainties

Problem Specification Document (not for COMP101)

- Contains:
 - System Specification, Software Requirements, User-Needs Inventory
 - Purpose is to refine a rough statement into a complete and unambiguous statement of what it is to be solved
 - May be 4 sections in the document
 - 1 Input section
 - 2 Output section
 - 3 Exception Handling section
 - 4 Performance section
- 
- ‘functional specifications’ –
they describe the behaviour
expected for all combinations
of inputs

Specification Document

- The Problem Specification document is a 'Black Box' View
 - It specifies inputs and outputs but not the actual processing or data types needed to turn the inputs into the required outputs



Rough Statement

- You have been tasked to provide a program for a client who has submitted a request to:

“Record and average student test scores”

- Using input-process-output we can take it to a draft#2

“Each student takes three tests in a semester.

Input of three tests per student is via a terminal (VDU/keyboard).

Calculate final score for each student.

Output the test number, mean and the standard deviation”.

Specification: Questions to client

- Input
 - What units, values and level of accuracy?
 - Format of the scores (integer (55, 67), real(56.8, 39.7), letter (A, C, B+))
 - What is order of input?
 - For entire class enter all test1 scores, then all test2 scores etc
 - OR for each student enter test1 score then test2 score etc
 - How many tests are there (if not disclosed already)?
 - How many values for each test are there (when will we know when to stop)?
 - What is the legal range of values?
 - What class of input device will be used (keyboard, file, voice ...)?
 - How many students are in the class (some may not take a test)?

Specification: Questions to client

- Process
 - What results are to be output (mean, median, mode?)
- Output
 - What format is needed?
 - type (numeric, string), accuracy, units
 - Usual to use mean and standard deviation for this sort of analysis
 - Output as alphanumeric or graph
 - Results display?
 - spacing, headers, titles, layout
 - What class of device?
 - hardcopy, vdu, plotter, ...

Problem Specification – draft#1

Spec Document sections 1, 2

- There are 150 students and each takes three tests in the semester
- Scores are integer range 0 to 100
- Input the scores via a terminal (VDU and keyboard) by entering the 3 scores for student-1 followed by the 3 scores for student-2 etc., finishing with the 3 scores for student-150
- Output the mean and standard deviation as shown right

Test #	mean	Standard deviation
1	xxx.x	xx.x
2	xxx.x	xx.x
3	xxx.x	xx.x

Problem Specification - draft#2

Spec Doc sections 1, 2

refinements to draft#1 in bold – improves ‘usability’

Input number of students, lowest legal score, highest legal score)

- A maximum of **N** students each takes three tests in the semester
- Test scores are integers in the range **LOW = 0 to HIGH = 100**
- Input the scores via a terminal in the order:
 - The number of students, **N**, where **$1 \leq N \leq 150$** ;
 - The lowest legal test score (**LOW**);
 - The highest legal test score (**HIGH**) where **$HIGH \geq LOW$** .

Follow these 3 inputs with input for the 3 scores for student-1 followed by the 3 scores for student-2 etc., finishing with the 3 scores for **student-N all within the range LOW to HIGH.**

- Output the mean and standard deviation as per table (see draft#1) **with an extra line below the table: ‘Number of students = xxx’**

Problem Statement draft#3

Spec Document sections 1, 2

- We probably have sufficient refinement to move to the next stage
- **Program Design**
 - Using pseudocode

STOP

For information: **Reading only follows**

Following slides continue with more advanced approaches in refining the specification further

Problem Statement draft#3 extended refinement

Spec Doc sections 1, 2

- Make it work for any number of tests (improves 'generality')
 - Input from file
 - Include student names/ID
 - Any other statistical measures
 - Output in sorted sequence
 - Output graphs
 - etc

Problem Statement draft#3 extended

Spec Doc section 3: Exception handling

- Handle actions to take for all possible input
 - Implausible, illegal, impossible etc
 - Input outside of range
 - Too much input
 - Too little input
 - Input in wrong format
 - Input giving illegal operations (divide by 0)
 - Nobody took any of the tests ($N = 0$)
 - Too many candidates ($N > 150$)
 - Test score meaningless ($LOW > HIGH$)
 - Test score $\neq$ integer
 - Test score out of range (LOW to HIGH)

Problem Specification - draft#3 extended

Spec Doc section 3:Exception handling

In case of exceptions occurring:

- Output an error message and allow re-input
- Use default values
 - If user enters -5 then assume +5 and accept the entry
- Ignore the illegal entry so output will be based on N-1 values
- Terminate the program entirely, with error message

Problem Statement - draft#3 extended

Spec Doc section 3: Exception handling

- The 'Black Box' for this section now looks like this



Problem Statement Draft #3 extended

(section 3: refinements for exceptions to draft #2 in bold)

“A maximum of N students each takes three tests in the semester. Test scores are integers in the range LOW = 0 to HIGH = 100.

Via a terminal, input in order:

- a) The number of students, N where $1 \leq N \leq 150$;
- b) The lowest legal test score (LOW);
- c) The highest legal test score (HIGH) where $HIGH \geq LOW$.

If N, LOW or HIGH is out of range, print “Error: Input out of range – re-enter” and allow the user to re-enter a valid value

Follow these 3 inputs with input for the 3 scores for student-1 followed by the 3 scores for student-2 etc., finishing with the 3 scores for student-N all within the range LOW to HIGH.

If any test score is out of range, print “Error: test score out of range – re-enter”.

When all input is legally entered, output the mean and standard deviation and number of students as described previously”

Problem Statement Draft#3 end

- By now, this is looking like a workable specification
- We maintain a review for anything that has been overlooked
 - If a student is excused a test, then the score should not be included in the calculation
 - If 0 is entered against this student, the program will 'infinite loop' as the user will constantly be asked to re-enter a valid score!

Problem Statement - draft #4

Spec Doc Section 4: Performance

- The fourth Section is written for the user
 - reduce any technical issues
- Minimum operating system requirement
- Number of data sets per second that can be processed
- Performance metrics (must be unambiguous)
 - Average query response to be <2.0seconds
 - Single response to not be > 5.0seconds
 - 99% of data processed in < 10seconds
 - Metrics usually address: Speed, Throughput, Availability, Capacity, Precision, Equipment needed

Problem Specification Justification

- As no (full) coding has yet taken place, changes are easy to make
- In the stages that follow - design, coding and testing - changes are expensive in terms of cost and time
- Studies show it can be ten times more expensive to make changes after the specification stage
- Once the Problem Specification is signed off (it acts as a contract between user and developer), the user role is finished until delivery
- The next stage now begins – **PROGRAM DESIGN**
 - Use pseudocode for this problem